University of Edinburgh

School of Informatics

Internship report

M1 Informatique Concepts et Applications

Varieties of Semiring Solvers via Multicategorical and Algebraic Constructions

Intern:
Amaury Mazoyer
ENS de Lyon,
France

Supervisor:
Ohad Kammar
University of Edinburgh,
United Kingdom

May 4 – July 31 2026

1 Introduction

In theorem provers and dependently-typed programming languages, users often have to prove to a type checker that two terms are equivalent. Constructing the proof often involves repetitive application of the axioms defining the underlying theory. For instance, in order to prove formally the classic identity

$$\forall a, b \in \mathbb{R}, (a+b)^2 = a^2 + 2ab + b^2$$

one has to invoke 10 semiring axioms: associativity of the addition three times, commutativity of the multiplication once, left distributivity of times over plus three times, left neutrality of the multiplication twice and evaluation once.

In order to free users from needing to construct such proofs, most dependently typed languages include simplifiers for common algebraic structures. In particular, solvers for semirings and rings already exist and are implemented in most proof assistants. The state-of-the-art in terms of ring solvers is Rocq's current implementation of its `ring` tactic [Grégoire & Mahboubi, 2005]. This is also the implementation chosen by Agda [Kidney, 2019]. The technique used is called reflection, and while Grégoire and Mahboubi managed to unify the implementation for semirings and rings, it is limited to those theories. Therefore, one would need to implement a new solver from the ground up to deal with involutive rings for instance.

The goal of this internship is to find a way to combine existing solvers for component theories to obtain a solver for distributive combinations of theories. Doing so would let us implement a semiring solver from a monoid solver and a commutative monoid solver. Other than letting us implement solvers more easily, this also opens the door to a wide variety of other semiring solvers, considering combinations of varieties of semigroups, monoids and groups.

The concept used to implement such solvers is called **fr**ee **ex**tensions, abbreviated Frex. Introduced in Yallop et al. [2018], and built on universal algebra, it is first thought as a way to represent partially-static structures, where some parts of a data structure are known at compile-time (static) and other parts are known only at run-time (dynamic). Frex relies on *normal forms*, i.e., canonical representatives using the algebraic laws while also evaluating concrete elements. The first application of Frex was multi-stage programming (MSP), a form of metaprogramming where parts of source code are annotated and become available as symbolic expressions. Such staged code can be manipulated within the same program before it is translated at either run-time or compile time. This warrants the use of partially-static structures. Frex can also be specialised to fully dynamic structures, considering free algebras (abbreviated fral) instead of free extensions.

Later, Allais et al. found another use for Frex. With its use of normal forms, Frex represents terms modulo semantics. This leads to so-called 'representation theorems' which can be used for algebraic simplification thanks to universal algebra. Allais et al. implemented an extensible dependently typed library for algebraic simplifiers based on fral and frex representation theorems.

Frex can be extended modularly: Allais et al. [2023] describes a modular construction of an involutive algebra frex out of the frex of the underlying algebra. This lets them expand for free on already constructed free extensions, and combined with the work of Allais et al. [2025], this gives solvers for involutive structures for free. For instance, implementing a simple monoid solver using frex yields an involutive monoid solver.

This justifies the use of Frex to try to find a modular construction for solvers for distributive combination of theories. Doing so then amounts to finding ways to combine free extensions to form other free extensions.

1.1 Outline and contributions

In §2, we present the relevant mathematical concepts behind Frex, how they are used in practice to solve equations and also some concepts of category theory that are used to state the main result.

In §3, we present the theoretical construction of free algebras for the distributive combination of theories. Ohad Kammar had stated an incomplete result (§3.2), but it turns out it contradicted known results (§3.3). We combed the (incomplete) proof for errors, strengthened the hypotheses to state a new corrected result and proved it (§3.4). §3.5 aims at extending the result from free algebras to free extensions, but the construction is incomplete.

In §4, we briefly present the implementation of the distributive combination of free algebras in Idris 2, to what extent it was implemented, as well as a full performance evaluation (§4.4). To do so, we also had to implement some additional solvers for the component theories (§4.3).

In §5, we compare different aspects of Frex with the state-of-the-art in terms of semiring solvers: Rocq's `ring` tactic. §5.1 and §5.2 give insights into how the `ring` tactic works, while §5.3 explains the soundness and completeness guarantees of both libraries. §5.4 compares the capabilities and applications of Frex and the `ring` tactic.

The appendices contain some additional context in which this internship took place (§B), the Idris 2 source code for the whole project (§C), the full proof of Theorem 4 (§D) and a link to my research notes (§E).

2 Preliminaries

Frex is based on some concepts of *universal algebra*, which gives generic descriptions of algebraic structures and relations between them [Burris & Sankappanavar, 1981, Chp. II]. We'll summarise the relevant concepts, which have been formalised in Idris 2 [Allais et al., 2025]. We'll use the example of monoids to illustrate these concepts along the way. We'll also explain how the solving works, and we'll define the distributive combination of theories formally.

2.1 Syntax

A *finitary signature* $\Sigma = (\operatorname{op}\Sigma, \operatorname{arity})$ consists of a set $\operatorname{op}\Sigma$ of *operators*, and an assignment $\operatorname{arity} : \operatorname{op}\Sigma \to \mathbb{N}$ associating to each operator in $\operatorname{op}\Sigma$ its *arity*. The usual signature $\Sigma_{\mathbf{Mon}}$ of monoids consists of two operators $(\cdot)$ and 1, with respective arities 2 and 0. We'll write $\Sigma_{\mathbf{Mon}} \coloneqq \{(\cdot) : 2,\ 1 : 0\}$.

An *algebra* $A = (\underline{A}, A[\![-]\!])$ over a signature Σ consists of a set $\underline{A}$ called the *carrier* of A, and a function $A[\![f]\!] : \underline{A}^n \to \underline{A}$ called the *interpretation* of f in A for each operator $f : n \in \Sigma$. This gives the semantics to the algebraic language determined by the signature. There may be different algebras for a given signature and carrier set. One can equip the set $\mathbb{N}$ of natural numbers with the monoid structure $(\mathbb{N}, (+), 0), (\mathbb{N}, (\cdot), 0)$ or even $(\mathbb{N}, \max, 0)$.

Given a set X of variables and a signature Σ, the set of Σ-*terms over* X, denoted $\operatorname{Term}_\Sigma(X)$, is defined by induction:

$$\frac{x \in X}{\operatorname{var} x \in \operatorname{Term}_\Sigma(X)} \qquad \frac{t_i \in \operatorname{Term}_\Sigma(X) \quad f : n \in \Sigma}{f(t_1, \ldots, t_n) \in \operatorname{Term}_\Sigma(X)}$$

We'll usually write $x \in \operatorname{Term}_\Sigma(X)$ instead of $\operatorname{var} x \in \operatorname{Term}_\Sigma(X)$. We define *equations in context* $X \vdash l = r$ as triples (X, l, r) where X is a set of variables and l, r are terms in context X called

the *left-hand-side* (LHS) and *right-hand-side* (RHS). The associativity equation in $\Sigma_{\mathbf{Mon}}$ is then defined as $x, y, z \vdash x \cdot (y \cdot z) = (x \cdot y) \cdot z$.

An *environment* ρ for a context X, or X*-environment*, in an algebra A is a function $\rho : X \to \underline{A}$ that assigns to each variable in X an element of the carrier of A. Given a term $t \in \mathrm{Term}_\Sigma(X)$, denoted $X \vdash t$, the algebra A gives an *interpretation* function $A[\![t]\!] : \underline{A}^X \to \underline{A}$. For an X-environment ρ, $A[\![t]\!]\,\rho$ is defined by induction:

$$A[\![\mathrm{var}\, x]\!]\,\rho := \rho\, x \qquad A[\![\mathrm{op}(t_1, \ldots, t_n)]\!]\,\rho := A[\![\mathrm{op}]\!]\,(A[\![t_1]\!]\,\rho, \ldots, A[\![t_n]\!]\,\rho)$$

For example, in the $\Sigma_{\mathbf{Mon}}$-algebra $A = (\mathbb{N}, (+), 0)$, one can interpret the term $x \cdot (y \cdot 1)$ in the environment $\{x \mapsto 1, y \mapsto 2\}$, yielding $A[\![x \cdot (y \cdot 1)]\!] = 1 + (2 + 0) = 3$

An equation $X \vdash l = r$ between terms in $\mathrm{Term}_\Sigma(X)$ is *valid* in an algebra A if for every environment $\rho : X \to \underline{A}$, we have $A[\![l]\!]\,\rho = A[\![r]\!]\,\rho$. We write this $A \vDash (X \vdash l = r)$. For example, the $\Sigma_{\mathbf{Mon}}$ algebras presented so far validate the associativity axiom, but interpreting the binary operation as subtraction over the integers $(\mathbb{Z}, (-), 0)$ does not: taking the environment $\{x \mapsto 0,\ y \mapsto 0,\ z \mapsto 1\}$, we have $0 - (0 - 1) = 1 \neq -1 = (0 - 0) - 1$.

A *presentation* (or *theory*) $\mathcal{T} = \langle \Sigma_\mathcal{T}, \mathrm{Ax}_\mathcal{T} \rangle$ consists of a signature $\Sigma_\mathcal{T}$ and a set $\mathrm{Ax}_\mathcal{T}$ of Σ-equations in context. A $\mathcal{T}$*-model* A is a $\Sigma_\mathcal{T}$ algebra validating all of the axioms in $\mathrm{Ax}_\mathcal{T}$. The **Monoid** presentation consists of the associativity axiom paired with the neutrality axioms $x \vdash x \cdot 1 = x$ and $x \vdash 1 \cdot x = x$ over the signature $\Sigma_{\mathbf{Mon}}$.

Let $A = (\underline{A}, A[\![-]\!])$ and $B = (\underline{B}, B[\![-]\!])$ be algebras over the same signature Σ. A *homomorphism* of Σ-algebras from A to B is a function $h : \underline{A} \to \underline{B}$ such that for every operator $f : n \in \Sigma$ and every $a_1, \ldots, a_n \in \underline{A}$ we have:

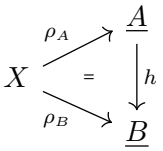
$$B[\![f]\!]\big(h(a_1), \ldots, h(a_n)\big) = h\big(A[\![f]\!]\,(a_1, \ldots, a_n)\big)$$

In other words, a homomorphism is a semantics-preserving function between the carriers of the algebras. For example, complex conjugation is a homomorphism between the $\Sigma_{\mathbf{Mon}}$-algebras over the complex numbers, since $\overline{z_1 + z_2} = \overline{z_1} + \overline{z_2}$, $\overline{0} = 0$ and $\overline{z_1 \cdot z_2} = \overline{z_1} \cdot \overline{z_2}$, $\overline{1} = 1$. A *homomorphism* of $\mathcal{T}$-models is a homomorphism of $\Sigma_\mathcal{T}$-algebras between the underlying algebras of the models.

2.2 Free models/algebras

Let $\mathcal{T}$ be a presentation and X a set of variables. A $\mathcal{T}$-model over X is just a pair $\langle A, e \rangle$ where A is a $\mathcal{T}$-model and $e : X \to \underline{A}$ is an X-environment in this algebra. This concept is used to formalise the fral solving process in §2.4.

A *morphism* $h : \langle A, \rho_A \rangle \to \langle B, \rho_B \rangle$ between $\mathcal{T}$-models over X is a $\mathcal{T}$-model homomorphism that makes the diagram on the right commute. A *free* $\mathcal{T}$*-model over* X is a $\mathcal{T}$-model $\langle F(X), \mathrm{env} \rangle$ such that for every $\mathcal{T}$-model A over X, there exists a unique morphism of $\mathcal{T}$-models from $\langle F(X), \mathrm{env} \rangle$ to A. We will often use the term "free algebra", abbreviated "fral", when $\mathcal{T}$ is implicit. The existence-and-uniqueness property is called the *universal property* of the free algebra. Thanks to the universal property, all free $\mathcal{T}$-models over X are isomorphic and therefore we often talk about *the* free algebra over X. Informally, the free algebra for a theory is the simplest and most general model validating the theory, in the sense that the only equations validated by the free algebra are the ones from the theory and the ones that we can deduce from them. This is ensured by the universal property. The free commutative monoid over the set of variables $\{x_1, x_2, x_3\}$ can be represented by $\mathbb{N}^3$. Addition is coordinate wise, the 0 element is $(0, 0, 0)$



and env is the function that sends x_1 to $(1,0,0)$, x_2 to $(0,1,0)$ and x_3 to $(0,0,1)$. The unique morphism out of this algebra into any commutative monoid A is the function sending (a_1,a_2,a_3) to $a_1(\rho_A(x_1))+a_2(\rho_A(x_2))+a_3(\rho_A(x_3))$. For monoids, the free monoid over a set of variables X can be represented as finite lists of elements of X, with concatenation as multiplication and where variables are injected as singleton lists.

Every presentation $\mathcal{T}$ induces an equivalence relation between terms using the deduction rules of equational logic. Explicitly, we define a *provability* relation $X \vdash l = r$ inductively as in Figure 1. Derivations are built similarly to natural deduction.

$$\frac{\mathrm{ax} \in \mathrm{Ax}_{\mathcal{T}}}{\mathrm{ax}}\ \text{(axiom)} \qquad X \vdash t = t\ \text{(reflexivity)} \qquad \frac{X \vdash l = r}{X \vdash r = l}\ \text{(symmetry)}$$

$$\frac{X \vdash l = m \quad X \vdash m = r}{X \vdash l = r}\ \text{(transitivity)} \qquad \frac{X \vdash l = r \quad \theta : X \to \mathrm{Term}_{\Sigma_{\mathcal{T}}} Y}{Y \vdash l[\theta] = r[\theta]}\ \text{(substitution)}$$

$$\frac{\text{for } i = 1,\dots,n : X \vdash l_i = r_i}{X \vdash \mathrm{op}(l_1,\dots,l_n) = \mathrm{op}(r_1,\dots,r_n)}\ \text{(congruence)}$$

Figure 1: Provability

Allais et al. [2025] proves that there is an isomorphism of $\mathcal{T}$-models over X

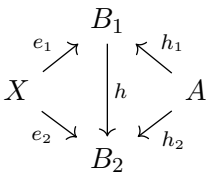
$$\langle F(X), \mathrm{env}\rangle \cong \left\langle \left(\mathrm{Term}_{\Sigma_{\mathcal{T}}} X\right)/\left(X \vdash_{\mathcal{T}}\right) =: Q, \mathrm{env}'\right\rangle$$

with $\mathrm{env}' : \left| \begin{array}{ll} X \to & Q \\ x \mapsto & [\mathrm{var}\, x] \end{array} \right.$. Therefore free algebras represent terms modulo semantics.

2.3 Free extensions

An *extension of A by X* is a triple $\langle B, h, e\rangle$ where B is a $\mathcal{T}$-algebra, $h : A \to B$ is a $\mathcal{T}$-homomorphism and $e : X \to B$ is a function.

Let $b_1 = \langle B_1, h_1, e_1\rangle, b_2 = \langle B_2, h_2, e_2\rangle$ be two extensions of A by X. A *morphism of extensions* $h : b_1 \to b_2$ is a $\mathcal{T}$-homomorphism $h : B_1 \to B_2$ that makes the diagram on the right commute. An extension $\langle A[X], \mathrm{sta}, \mathrm{dyn}\rangle$ is *free* when, for every extension $\langle B, h, e\rangle$, there is a unique extension morphism $[B,h,e] : \langle A[X], \mathrm{sta}, \mathrm{dyn}\rangle \to \langle B, h, e\rangle$. Free extension is abbreviated "frex", and a free extension of A by X is usually denoted $A[X]$. Just like for free algebras, all free extensions of A by X are isomorphic to each other. Let A be a commutative monoid. Its frex by the finite set of variables $\{x_1, x_2, x_3\}$ can be represented by $A[X] := \underline{A} \times \mathbb{N}^3$. The remaining structure is defined by:



$$\langle a, \langle k_{11}, k_{12}, k_{13}\rangle\rangle + \langle b, \langle k_{21}, k_{22}, k_{23}\rangle\rangle := \langle a+b, \langle k_{11}+k_{21}, k_{12}+k_{22}, k_{13}+k_{23}\rangle\rangle$$

$$0 := \langle 0, \langle 0,0,0\rangle\rangle \qquad \mathrm{sta}\ a := \langle a, \langle 0,0,0\rangle\rangle \qquad \mathrm{dyn}\ x_1 := \langle 0, \langle 1,0,0\rangle\rangle \qquad \mathrm{dyn}\ x_i := \dots$$

$$[B,h,e]\ \langle a, \langle k_1,k_2,k_3\rangle\rangle := h\ a + k_1 \cdot e\ x_1 + k_2 \cdot e\ x_2 + k_3 \cdot e\ x_3$$

Similarly to §2.2, there is a canonical isomorphism between any free extension and the quotient frex $\left\langle \left(\mathrm{Term}_{\Sigma_{\mathcal{T}_A}} X\right)/\left(X \vdash_{\mathcal{T}_A}\right) =: Q, \mathrm{sta}', \mathrm{dyn}'\right\rangle$ with $\mathrm{sta}' : \left| \begin{array}{ll} A \to & Q \\ a \mapsto & [\underline{a}] \end{array} \right.$ and $\mathrm{dyn}' : \left| \begin{array}{ll} X \to & Q \\ x \mapsto & [\mathrm{var}\, x] \end{array} \right.$. $\mathcal{T}_A$ is the presentation where the operators are the ones of $\mathcal{T}$ together with newly adjoined

constants $\underline{a}$ for every $a \in \underline{A}$, and whose axioms are the combination of the axioms in $\mathcal{T}$ and the evaluation axioms $\vdash \mathrm{op}(\underline{a_1}, \ldots, \underline{a_n}) = A[\![\mathrm{op}]\!](a_1, \ldots, a_n)$ for every $\mathrm{op} : n \in \Sigma_{\mathcal{T}}$ and $(a_1, \ldots, a_n) \in \underline{A^n}$ [Allais et al., 2023]. In this way, each free extension is also a free algebra of a theory specialised to a concrete algebra by adding axioms over the concrete elements.

2.4 How fral/frex solving works

We explain how fral solving works, and then how to adapt it to free extensions. This has been formalised in Idris 2 [Allais et al., 2025], see §4.

Suppose that we want to prove that the equation $x + (y + x) + y = x + x + y + y$ holds in a commutative monoid A. We evaluate the LHS and RHS of the equation in the free algebra, both give the representative $(2, 2)$. We then prove the equality in the free algebra, this is simple reflexivity here, and we use the universal property of the free algebra to conclude that the equation is valid in A.

In the general setting, let X be a set of variables, $\mathcal{T}$ be a theory, $\langle A, \mathrm{env}_A \rangle$ be a $\mathcal{T}$-model over X and $\langle FX, \mathrm{env}_{FX} \rangle$ be a free $\mathcal{T}$-model over X. We have the following proposition

Proposition 1

Let eq := $X \vdash l = r$ be an equation in context.

If the free algebra FX validates eq then A validates eq.

Proof

This is proved in §A. □

Proving equalities in arbitrary models with free algebras then just comes down to proving equalities in the free algebra. Since free algebras are canonicaly isomorphic to the quotient algebra of terms quotiented by the provability relation, if an equality is indeed provable, then the two associated terms in the free algebra should also be equal. The only difficulty lies in the definition of equality in the free algebra.

Proving equalities between partially static terms using free extensions is similar and reuses the same construction. We use the fact that a free extension is a free algebra for a specific theory as mentioned in §2.3. Instead of considering an equation on X, we consider an equation on $X \sqcup \underline{A}$. For an extension $\langle A, \mathrm{sta}, \mathrm{dyn} \rangle$, the environment we use is defined as follows:

$$\mathrm{env}\ x := \begin{cases} \mathrm{sta}\ x & \text{if } x \in \underline{A} \\ \mathrm{dyn}\ x & \text{if } x \in X \end{cases}$$

Using the same reasoning as before and the universal property of the free extension, we can prove partially static equalities on arbitrary models.

2.5 The distributive combination of theories

Given two signatures Σ_1 and Σ_2, we define their coproduct $\Sigma_1 + \Sigma_2$ as the signature with operators the disjoint union of the operators, but retaining their arity:

$$\Sigma_1 + \Sigma_2 = \{\iota_i f : n \mid (f : n) \in \Sigma_i, i = 1, 2\}$$

Given a Σ_i-term-in-context $n \vdash_{\Sigma_i} t$, we denote by $n \vdash_{\Sigma_1 + \Sigma_2} \iota_i[t]$ the $\Sigma_1 + \Sigma_2$-term-in-context obtained by applying the injection ι_i to all the operators of t. Similarly, given a Σ_i-equation in

context $\mathrm{eq} := (n \vdash_{\Sigma_i} t = t')$, we denote by $\iota_i[\mathrm{eq}]$ the $\Sigma_1 + \Sigma_2$-equation in context $n \vdash_{\Sigma_1+\Sigma_2} \iota_i[t] = \iota_i[t']$.

Let $\mathcal{A}$ and $\mathcal{M}$ be two theories. We define the *distributive combination of $\mathcal{M}$ over $\mathcal{A}$* [Hyland & Power, 2006] to be the theory $\mathcal{M} \triangleright \mathcal{A}$ given by:

$$\begin{aligned}
\Sigma_{\mathcal{M}\triangleright A} &:= \Sigma_{\mathcal{M}} + \Sigma_{\mathcal{A}} \\
\mathrm{Ax}_{\mathcal{M}\triangleright A} &:= \iota_1[\mathrm{Ax}_{\mathcal{M}}] \cup \iota_2[\mathrm{Ax}_{\mathcal{A}}] \\
&\quad \cup \left\{ \begin{array}{rl} \langle x_i \rangle_{\substack{i=1 \\ i\neq i_0}}^{n}, \mathbf{y} \mid f(x_1, \ldots, x_{i_0-1}, s\mathbf{y}, x_{i_0+1}, \ldots, x_n) & \\ \qquad = s\left\langle f(x_1, \ldots, x_{i_0-1}, \mathbf{y}_j, x_{i_0+1}, \ldots, x_n) \right\rangle_{j=1}^{|\mathbf{y}|} & \end{array} \middle| \begin{array}{l} f : n \in \Sigma_{\mathcal{M}} \\ s : |\mathbf{y}| \in \Sigma_{\mathcal{A}} \end{array} \right\}
\end{aligned}$$

Concretely, the signature is the coproduct signature and the axioms are the union of the axioms to which we added distributivity axioms between the operators of $\mathcal{M}$ and $\mathcal{A}$, stating that every operator of $\mathcal{M}$ distributes over every operator of $\mathcal{A}$. We will call $\mathcal{M}$ the multiplicative theory, and $\mathcal{A}$ the additive theory. For example, the theory of semirings is the distributive combination of commutative monoids over monoids. The distributivity axioms when unfolded state that:

$$\begin{aligned}
x, y, z \vdash x \cdot (y + z) = (x \cdot y) + (x \cdot z) \qquad & x, y, z \vdash (x + y) \cdot z = (x \cdot z) + (y \cdot z) \\
x \vdash x \cdot 0 = 0 \qquad & x \vdash 0 \cdot x = 0
\end{aligned}$$

The multiplicative operation acts linearly in each argument with respect to the additive theory, and constants of the additive theory taken as operators give annihilation laws.

2.6 Monads and distributive laws

Monads are closely related to algebraic theories, and the construction of the distributive combinations in §3 uses the notions of monads and distributive laws from a category-theoretical point of view. We will assume basic knowledge about category theory, mainly about functors, natural transformations and adjunctions. One can refer to MacLane [1971].

A *monad* on a category $\mathcal{C}$ is a triple (T, η, μ) where $T : \mathcal{C} \to \mathcal{C}$ is a functor, $\eta : \mathrm{Id}_{\mathcal{C}} \to T$ is a natural transformation called the unit, $\mu : T^2 \to T$ is a natural transformation called the multiplication, and such that the following diagrams commute:

$$\begin{array}{ccc} T & \xrightarrow{T\eta} & T^2 \\ {\scriptstyle \eta T}\downarrow & \searrow & \downarrow{\scriptstyle \mu} \\ T^2 & \xrightarrow[\mu]{} & T \end{array} \qquad \begin{array}{ccc} T^3 & \xrightarrow{T\mu} & T^2 \\ {\scriptstyle \mu T}\downarrow & & \downarrow{\scriptstyle \mu} \\ T^2 & \xrightarrow[\mu]{} & T \end{array}$$

Every adjunction $\langle F, G, \eta, \varepsilon \rangle$ gives rise to a monad [MacLane, 1971], by defining $T := GF$, $\eta := \eta$ and $\mu := G\varepsilon F$.

Theorem 2 (Zwart and Marsden, 2018)

For a theory $\mathcal{T}$, there is a left adjoint $F^{\mathcal{T}}$ to the obvious forgetful functor $U^{\mathcal{T}} : \mathcal{T}\text{-}\mathbf{Alg} \to \mathbf{Set}$ where $\mathcal{T}\text{-}\mathbf{Alg}$ is the category with $\mathcal{T}$-models as objects and $\mathcal{T}$-model homomorphisms as morphisms. The functor is defined as follows:

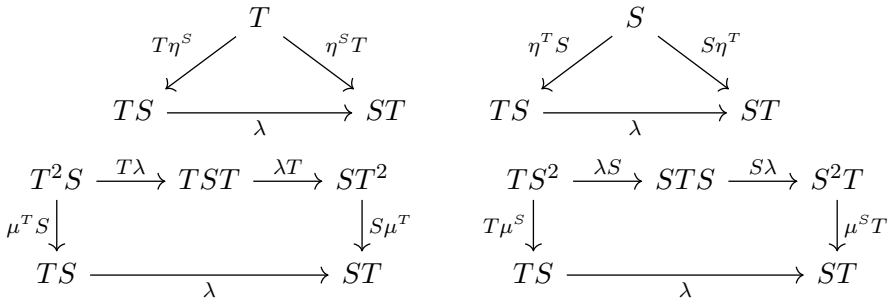
- For a set X, $F^{\mathcal{T}}X := \underline{F_{\mathcal{T}}X}$ is the carrier of the free $\mathcal{T}$-algebra over X.
- The action on a function $h : X \to Y$ is defined inductively by:

$$F^{\mathcal{T}}(h)(x) = h(x) \quad \text{for } x \in X$$
$$F^{\mathcal{T}}(h)(\mathrm{op}(t_1, \ldots, t_n)) = \mathrm{op}\big(F^{\mathcal{T}}(h)(t_1), \ldots, F^{\mathcal{T}}(h)(t_n)\big) \quad \text{for } \mathrm{op} : n \in \Sigma_{\mathcal{T}}$$

For a theory $\mathcal{T}$, the *free model monad* induced by $\mathcal{T}$ is then the monad induced by the free/forgetful adjunction $U^{\mathcal{T}} \circ F^{\mathcal{T}}$. We say that a monad T corresponds to a theory $\mathcal{T}$ if T is a free model monad induced by $\mathcal{T}$. It is well known that every finitary monad on the category of sets arises as a free model monad for some algebraic theory.

Concretely, the monad T corresponding to the algebraic theory $\mathcal{T}$ is the monad whose terms TX are terms over the variables in X modulo the equations, whose unit $\eta_X : X \to TX$ sends each variable to the corresponding atomic term and whose multiplication $\mu_X : TTX \to TX$ is substitution/flattening, i.e, the act of plugging terms into terms. For the theory of (regular) monoids, the corresponding monad is the List monad whose elements LX are finite lists over X, $\eta^L_X(x) = [x]$ and μ^L_X is list concatenation.

Given two monads (T, η^T, μ^T) and (S, η^S, μ^S) on a category $\mathcal{C}$, a *distributive law* [Beck, 1969] of T over S is a natural transformation $\lambda : TS \to ST$ such that the following diagrams commute:



These equations ensure that the distributive law is compatible with both the units and the multiplications of S and T. This law induces a composite monad ST on $\mathcal{C}$ with unit $\eta^{ST} = \eta^S \eta^T$ and multiplication $STST \xrightarrow{S\lambda T} SSTT \xrightarrow{\mu^S \mu^T} ST$.

The most famous example of a distributive law involves the List monad distributing over the Abelian group monad (Definition 7): $\lambda : LA \Rightarrow AL$ [Beck, 1969]. It captures the classic distributivity of multiplication over addition. Simply put,

$$\lambda\big(a \cdot (b + c)\big) = a \cdot b + a \cdot c$$

We can write lists as formal products and multisets as formal sums, letting us write the following definition for λ:

$$\lambda\left(\prod_{i=0}^{n} \sum_{j_i=0}^{m_i} x_{\langle i, j_i \rangle}\right) = \sum_{j_0=0}^{m_0} \cdots \sum_{j_n=0}^{m_n} \prod_{i=0}^{n} x_{\langle i, j_i \rangle}$$

The resulting composite monad is the ring monad, i.e the monad corresponding to the algebraic theory of rings. This is the essence of what we will use to construct the free algebra for the distributive combination of theories.

3 Categorical approach to the construction of distributive combinations

While using Frex does not require knowledge about category theory, Frex uses many category-theoretic concepts internally. Therefore, the construction of distributive combinations is primarily category-theoretic. However, we do not go into the details here, as this would require a course on 2-categories and multicategories.

3.1 Challenge about constructing distributive combinations

Constructing generic free algebras and free extensions for arbitrary theories is not hard. In fact, §2.2 gives an explicit construction for the free algebra of any theory using quotients. However in order to solve equations computationally, §2.4 shows that we need a normal form with decidable equality, not merely an abstract quotient construction. The equivalence relation between terms in the quotient is not necessarily effective/decidable. Without this, we won't be able to prove equalities in the free algebra, and therefore to prove any equation with free algebras.

Free extensions also have a generic construction: Allais et al. [2025] and Yallop et al. [2018] state that the free extension of an algebra A by X is the category-theoretic coproduct of A with the free $\mathcal{A}$-algebra over X. However, there is no generic formula for the coproduct of two algebras for an arbitrary theory.

The difficulty of the construction is for it to be effective so that equivalence between two terms can be determined computationally. This forbids the use of quotients, category theory pushouts etc...

3.2 Initial result on the free algebra for the distributive combination

A first result was given by Ohad Kammar (my internship supervisor) about the free algebra for the distributive combination of a theory over a commutative theory. A theory $\mathcal{A}$ is commutative iff every operation of the theory commutes with every other operations, or more formally, iff for every $f : n \in \Sigma_{\mathcal{A}}$ and every $g : m \in \Sigma_{\mathcal{A}}$,

$$f\begin{pmatrix} g(x_{11},\dots,x_{1m}) \\ \vdots \\ g(x_{n1},\dots,x_{nm}) \end{pmatrix} = g\left(f\begin{pmatrix} x_{11} \\ \vdots \\ x_{n1} \end{pmatrix} \quad \cdots \quad f\begin{pmatrix} x_{1m} \\ \vdots \\ x_{nm} \end{pmatrix}\right) \tag{1}$$

This definition is stronger and more structural than ordinary commutativity of a single binary operation. For monoids, this states that

$$(x \cdot y) \cdot (z \cdot w) = (x \cdot z) \cdot (y \cdot w) \qquad 0 \cdot 0 = 0 \qquad 0 = 0$$

The theory of commutative monoids is therefore a commutative theory, fortunately. Note that every operator of arity 0 and 1 commutes with itself.

The initial result was formulated in terms of multicategories, we reformulate it here in terms of monads for ease of understanding.

Theorem 3 (Free algebra for the distributive combination, incomplete)

Let $\mathcal{A}$ be a commutative theory, and $\mathcal{M}$ be any theory. Then there exists a distributive law of the free model monad induced by $\mathcal{M}$ over the free model monad induced by $\mathcal{A}$, and the free model monad induced by $\mathcal{M} \triangleright \mathcal{A}$ is the composite monad of the two free model monads.

In the semiring case, this says that if $F_{\mathcal{M}}X$ gives multiplicative normal forms and $F_{\mathcal{A}}X$ gives additive normal forms, then $F_{\mathcal{A}}F_{\mathcal{M}}X$ gives normal forms for the semiring over X.

Intuitively, the theorem says that given two free algebras for the additive and multiplicative part, composing them gives the free algebra for the distributive combination of the theories. This construction is effective in the sense of §3.1.

Even though the construction itself was well-defined, the proof wasn't complete, and for a good reason: the statement is incorrect. The culprit is the absence of distributive laws under some hypothesis.

3.3 Why the initial result fails

Zwart and Marsden [2018] gives multiple results about the absence of distributive laws under some hypothesis. We can regroup these results under 3 categories:

1. theorems 3.5, 3.12 and 4.24 all forbid the existence of an idempotent operator in the multiplicative theory, on top of other conditions.
2. theorem 4.7 forbids the existence of two distinct constants in the additive theory.
3. theorem 4.17 imposes the abides equation $(a * b) * (c * d) = (a * c) * (b * d)$ to hold for every operator $*$ in the additive theory.

The second and third point are both taken care of by the hypothesis of commutativity of the additive theory in Theorem 3. For the second point, two constants commute with each other if they are equal. A commutative theory can therefore not have two or more distinct constants. For the third point, the abides equation is simply the commutativity hypothesis for $f = g = *$.

However the first point is not accounted for. Informally, idempotence duplicates information in a way incompatible with the intended distributive-law construction. Under the current hypothesis, idempotent operators are allowed. Therefore, we need to strengthen our hypothesis.

A known failure is the distributive combination of semi-lattices with probabilistic nondeterminism [Varacca & Winskel, 2006]. Let $\mathcal{M} = \mathbf{BA}$ be the theory of barycentric algebras (Definition 8), $\mathcal{A} = \mathbf{SL}$ be the theory of join-semilattices (Definition 9). Barycentric algebras have idempotent operators, therefore there cannot exist a distributive law $\lambda : \mathbf{BA}\ \mathbf{SL} \Rightarrow \mathbf{SL}\ \mathbf{BA}$. The free semilattice on a set X is $F_{\mathbf{SL}}X = \mathcal{P}^+_f(X)$ the set of non-empty finite subsets of X, with union as join, and the free barycentric algebra on X is $F_{\mathbf{BA}}X = \mathcal{D}_f(X)$ the set of finitely-supported probability distributions on X. However, the free algebra on X for the distributive combination of the two is $\mathrm{Conv}_f(\mathcal{D}_f(X))$, the set of non-empty finitely generated convex subset of $\mathcal{D}_f(X)$, and not $\mathcal{P}^+_f(\mathcal{D}_f(X))$, the set of non-empty finite subsets of $\mathcal{D}_f(X)$.

3.4 Corrected result

We strengthen the hypothesis on $\mathcal{M}$ by requiring it to be an affine theory. A theory $\mathcal{M}$ is *affine* if every axiom is an equation between terms where each variable occurs at most once. This

allows weakening/deletion of variables, but not contraction. For example, $x, y \vdash x + y = y + x$ or $x \vdash x \cdot 0 = 0$ are affine equations, but $x \vdash x \vee x = x$ is not. Semigroups, monoids, groups and their commutative variants are all affine theories, whereas semilattices are not. This effectively covers the first point of the previous section.

We now state the corrected theorem, in terms of monads once again instead of multicategories.

Theorem 4 (Free algebra for the distributive combination)
Let $\mathcal{A}$ be a commutative theory, and $\mathcal{M}$ be an affine theory. Then there exists a distributive law of the free model monad induced by $\mathcal{M}$ over the free model monad induced by $\mathcal{A}$, and the free model monad induced by $\mathcal{M} \triangleright \mathcal{A}$ is the composite monad of the two free model monads.

The actual construction uses operads, translations from operads to theories, 2-categories and multicategories. Therefore, we won't present it nor the proof as is, but we'll explain the gist of it.

First, start by noticing that a model for the distributive combination is made of a $\mathcal{M}$-model and a $\mathcal{A}$-model on the same carrier set (this ensures that the respective axioms hold) where moreover the operations of $\mathcal{M}$ distribute over the operations of $\mathcal{A}$ (this is a pullback in the category of categories). Let X be a set of variables. The construction start by taking the free $\mathcal{M}$-algebra over X, $A_0 := F_{\mathcal{M}}X$. We then take the free $\mathcal{A}$-algebra over $\underline{F_{\mathcal{M}}X}$, $A_1 := F_{\mathcal{A}}\left(\underline{F_{\mathcal{M}}X}\right)$. This gives us our candidate $\mathcal{A}$-model. This step corresponds to the monadic composition, minus the multiplications. But we now need to define the semantics of the operators of $\mathcal{M}$ on the carrier $\underline{A_1}$. To do this, we "lift" the $\mathcal{M}$-operators so that the lifted operators on the new carrier distribute over the $\mathcal{A}$-operators. This is what gives the distributive law. Through a more involved argument, and with the help of the affine hypothesis, we prove that the $\mathcal{M}$-axioms hold. The commutativity hypothesis of $\mathcal{A}$ is used to prove the existence of an adjunction between multicategories, which is used to lift the $\mathcal{M}$-operators.

this probably deserves an exemple but I don't know how to present it

This construction is effective: it is solely built from applying the free algebra constructions and some 2-functors in well-chosen categories. The lifted operators can be defined concretely.

The description of the construction may not sound formal, but this is because we had to omit a lot of details for it to be understandable without the full multicategory and 2-category background.

3.5 An attempt at constructing the free extension for the distributive combination

The natural next step is to extend this result to free extensions. However the task is not so easy.

Since we have seen in §2.3 that a free extension is also a free algebra for a specialised theory, the first thought is to reuse our construction on this theory. However, the hypotheses do not hold anymore: the specialised theory contains a constant for every concrete element. As stated in §3.3, two constants commute with each other iff they are equal, and so the specialised theory for the free extension of any non-trivial algebra is not commutative. Therefore, we need to find a completely new construction.

Although the construction is nontrivial, it is not out of reach. We managed to define a construction using category-theoretic pushouts and the free extensions of the component theories. Although it is not effective, because pushouts are quotient-based, it shows that the free ex-

tensions of the component theories can indeed be used to construct the free extension of the combined theory. This led us to search for ways to adapt the construction for free algebras to free extensions.

The construction of free extensions for distributive combinations is still ongoing work. We figured out that we need to find a new pullback that is suited for extensions, and that we need to find a new adjunction between the multicategory of extensions and the obvious forgetful functor.

this is not actually correct stated as is, it mixes categories and functors

4 Implementation

Frex has multiple implementations, first in Haskell and MetaOCaml [Yallop et al., 2018], then in Agda [Corbyn, 2021] and finally in Idris 2 [Allais et al., 2025]. We are interested in the implementation in Idris 2, a purely functional programming language with first class types.

4.1 Setoids

Frex uses *setoids* [Bishop, 1967; Hofmann, 1997] instead of simple sets. This offers them some additional functionalities on top of term simplification and equation solving: printing terms and proofs, proof simplification, code generation and more [Allais et al., 2025].

A *setoid* is a set X equipped with an equivalence relation $\sim_X$. A *setoid homomorphism* $f : X \to Y$ is a function compatible with the equivalence relation: if $x \sim_X y$ then $f(x) \sim_X f(y)$. Every set X can be seen as a setoid by equipping it with the equality relation $(=)$. However setoids let us define some more complicated equivalence relations. For example, we can consider the setoids of terms for some theory $\mathcal{T}$ and set of variables X, equipped with the provability relation from Fig 1. Two equivalent terms in this setoid represent different, but provably equal, terms. This is different from the quotient $\big(\mathrm{Term}_{\Sigma_\mathcal{T}} X\big)/\big(X \vdash_\mathcal{T}\big)$ where elements represent equivalence classes of provably equal terms.

Frex implements a slightly different version of the universal algebra presented in §2. Carriers for algebras are setoids instead of sets. Therefore, all homomorphisms must also be setoid homomorphisms.

The use of setoids may not seem justified yet, but it enables functionalities beyond algebraic solving in Frex, such as proof pretty-printing, code generation and more, which is explained in §5.4.3.

maybe explain a bit more to justify?

4.2 Implementing the distributive combination

The distributive combination for free algebras has been implemented modularly so that the user only needs to provide free algebras for the component theories to get a free algebra for the combined theory. These free algebras can be passed to dedicated functions of the Idris Frex library to act as solvers.

Unfortunately, not the entire distributive combination of free algebras has been implemented in this internship. Only the semantics have been implemented, it remains to prove in Idris 2 that the constructed algebra validates the axioms and that it is indeed free. This is due to the fact that formalising such a conceptual proof that stems directly from category theory is long and tedious. However, since we do have the full theoretical proof, we already have some guarantees about our result and we will be able to implement the remaining proofs, however long this may take.

However, these proofs are not needed for the actual solving logic to work; they merely guarantee that the solver behaves as intended. We were therefore able to use the current implementation to prove equations on various varieties of semirings. For instance, for every example of a distributive combination we implemented (see the Table 1), we proved that all of the axioms of the combination hold. This also lets us test the performance of our implementation in §4.4. But before being able to run some tests, we had to implement the missing solvers for the component theories, as only solvers for monoids and commutative monoids were implemented in Idris 2.

4.3 Implementing solvers

Implementing solvers for a given algebraic theory amounts to finding the free algebra or free extension for that theory and formalising it in Idris.

We implemented fral solvers for semigroups, commutative semigroups and abelian groups. Solvers for monoids, commutative monoids and involutive monoids were already implemented. For similar reasons as in §4.2, we only implemented the semantics of the free algebras and left the proofs for future work. This is enough to use these solvers to solve equations.

The free algebras for these theories are well-known, therefore implementing fral solvers is just a matter of formalising these objects in Idris. Free semigroups are non-empty lists, free commutative semigroups are finite multisets with non-empty support and free abelian groups are integer-valued finitely supported maps.

4.4 Performance evaluation

Following the work of Allais et al., 2025, we tested our implementation to find the instantaneity (0.1s), interactivity (1s) and attention (10s) thresholds described by Nielsen, 1994. This is important as developing in Idris2 is interactive and our tool is meant to discharge the user of painful arithmetic proofs. Each datapoint in Figure 2 is the median execution time on 10 random terms of that size. Term size is computed as the number of leaves in the syntax tree of the term.

Frex's type-checking time[1] remains below the instantaneity threshold for terms of size 60 to 120, depending on the number of free variables, as pictured in Figure 2. This is more than sufficient to deal with the typical equalities that arise in dependently typed programs.

Frex eventually crosses the interactivity treshold as the term sizes grow. However, by that point, we notice that some datapoints are missing. This happens because the test machine ran out of RAM on some of the test cases. This indicates that Frex's bottleneck is not its speed but Idris itself, and suggests that Frex is likely suitable for practical applications.

Comparing the execution times for semirings and rings with the ones in Allais et al. [2025] for monoids, the performance seems to have much improved. This can be explained by two different factors:

- Allais et al. [2025] mentioned a performance bottleneck in Idris2's evaluator, this was four years ago and the development of Idris2 probably eliminated this bottleneck.
- The tests have not been run on the same machine, and the CPU used for the semirings test is much more efficient than the one used for the monoids tests.

[1] We used an Intel Core Ultra 7 255HX machine with 16GB memory, running Idris 2 version 0.8.0-6a54860ee on Debian Linux 13.

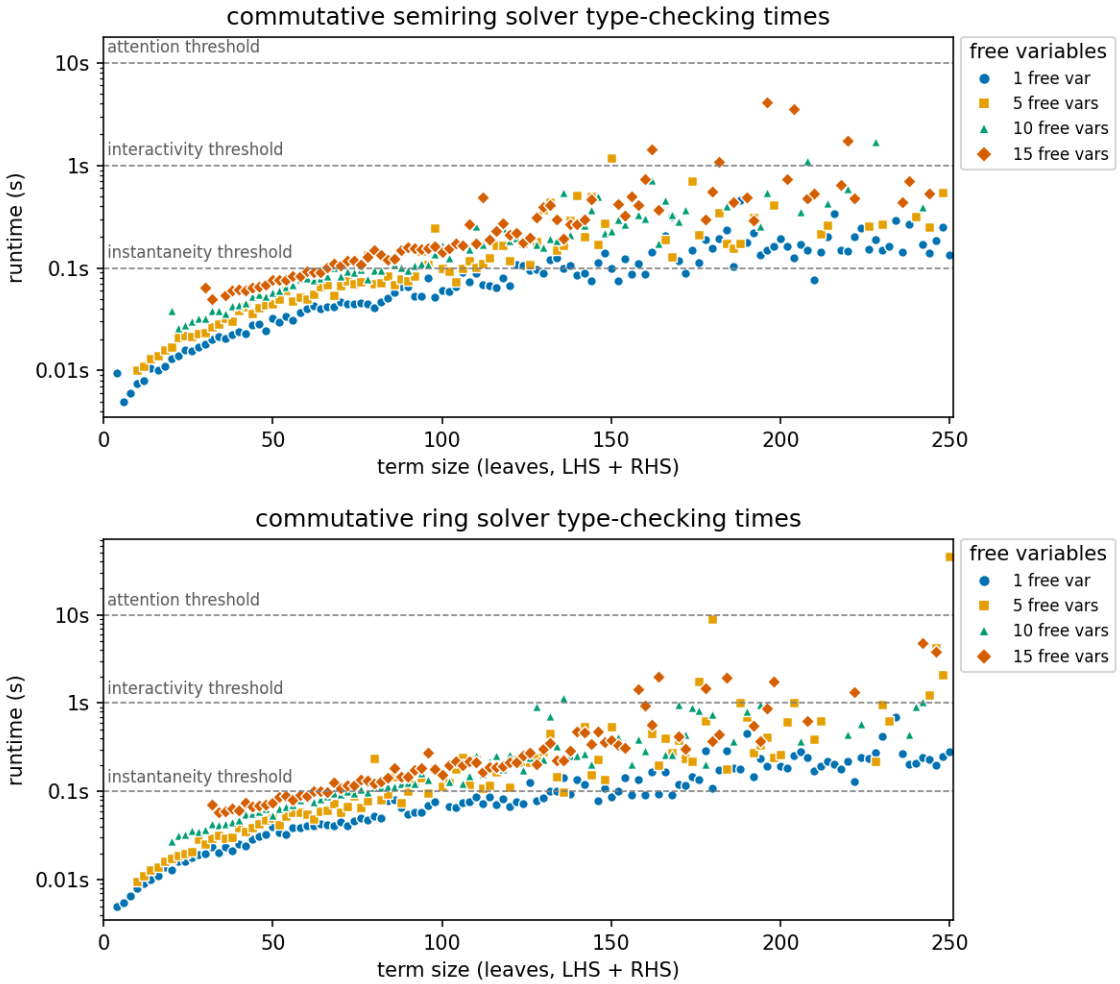


Figure 2: Type-checking times for the commutative semiring and ring solvers.

5 Comparison with existing solvers

In dependently-typed programming languages, the state-of-the-art for polynomial equalities over commutative semirings/rings was originally presented by Grégoire and Mahboubi [2005]. This is the current implementation of Rocq's `ring` tactic, and it was also adopted by Agda's ring solver [Kidney, 2019]. We'll explain briefly how the two approaches differ, and then compare the capabilities of both algorithms.

5.1 Reflection

Reflection is using the ability of the Calculus of Constructions to speak and reason about itself. Here is the pipeline for the reflection process used in Rocq described by Grégoire and Mahboubi [2005].

Consider a ring structure A, with two constants 0 and 1, three binary operators $+$, $*$ and $-$ and an opposite unary function $-$. We want to prove the equality of two terms t_1 and t_2 modulo the ring axioms. To do so, they introduce two inductive types PolExpr and Pol, two evaluation functions $\varphi_P : \text{PolExpr} \to A$ and $\varphi_{PE} : \text{Pol} \to A$ and a translation function $\mathfrak{T} : A \to \text{PolExpr}$. PolExpr describes the syntax of terms of type A, while Pol describes normal forms of terms of type A. The reflection process is pictured in Figure 3. It goes as follows:

1. from t_1 and t_2, build two corresponding terms e_1 and e_2 with $\mathfrak{T}$.

2. normalise these two polynomials using norm

3. prove the equality $\mathrm{norm}(e_1) = \mathrm{norm}(e_2)$ to conclude for the equality t_1 and t_2.

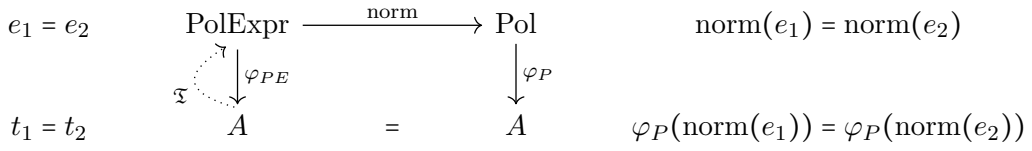


Figure 3: Reflection process

In order for this process to be correct, there are some constraints on φ_P, φ_{PE} and $\mathfrak{T}$ that need to be met:

$$\forall e \in \mathrm{PolExpr}, \varphi_{PE}(e) = \varphi_P(\mathrm{norm}(e)) \tag{2}$$

$$\forall a \in A, \varphi_{PE}(\mathfrak{T}(a)) = a \tag{3}$$

If the equality of the normal forms hold, it follows that:

$$t_1 = \varphi_{PE}(\mathfrak{T}(t_1)) = \varphi_P(\mathrm{norm}(\mathfrak{T}(t_1))) = \varphi_P(\mathrm{norm}(\mathfrak{T}(t_2))) = \varphi_{PE}(\mathfrak{T}(t_2)) = t_2$$

The type Pol is built so that equality can be checked syntactically: if two polynomials $P_1, P_2 \in \mathrm{Pol}$ are equal then so are their syntax tree. Therefore, equality of normal form comes for free and one only needs to prove the correctness criterion (2) for the tactic to be correct.

5.2 Almost-rings

The `ring` tactic is capable of dealing both with commutative rings and commutative semirings, under the same implementation. To unify both structure, they resort to *almost-rings*, an intermediate structure similar to rings and semirings.

An *almost-ring* is a commutative semiring completed with an unary operator called *almost-opposite*. The almost-opposite, denoted by $-$, satisfies the following axioms:

$$x, y \vdash -(x * y) = -x * y$$
$$x, y \vdash -(x + y) = -x + -y$$
$$x, y \vdash x - y = x + -y$$

The last equation defines an associated *pseudo-subtraction*.

These axioms do not allow to prove the missing axiom defining the opposite in a ring $x \vdash x + -x = 0$. This lets them equip a commutative semiring with an almost-ring structure by taking the almost-opposite to be the identity function, which satisfies the axioms. $\mathbb{N}$ can be seen as an almost-ring by defining $-x := x$ and $x - y := x + y$. Therefore, implementing a solver for almost-rings is sufficient to deal with both commutative semirings and commutative rings.

Even though the equation $x \vdash x + -x = 0$ is not provable in an almost-ring, it will be proved for rings by the `ring` tactic as long as $1 + (-1)$ reduces to 0 in the coefficient set used in the types Pol and $\mathrm{PolExpr}$.

5.3 Soundness and completeness

Both the Frex solvers and the `ring` tactic are sound and complete, but with a subtlety.

Any fral/frex is canonically isomorphic to the quotient fral/frex, and an effective fral/frex with a constructive universality proof has an effective isomorphism to the quotient setoid. In this way, a simplifier using an effective fral/frex is constructively sound and complete [Allais et al., 2025]. This completeness is with respect to the underlying theory of the fral/frex, not with respect to all actually provable equalities in the model for which we are trying to prove an equality. For instance, considering the ring of booleans with logical or as addition and logical and as multiplication, the addition is idempotent: $\forall x : \text{Bool}, x \vee x = x$. Even though it holds, the equation is not a consequence of the ring axioms and therefore it is unable to be solved by a fral for rings. Since both addition and multiplication are idempotent for this ring, we are not under the hypothesis of Theorem 4 and therefore Frex cannot be used to solve every equation over the booleans.

The soundness of the `ring` tactic comes from the correctness lemma (2). `ring` computes the normal forms for the LHS and the RHS of the equation and checks whether they are equal or not. If they are equal, (2) assures us that the two terms are indeed equal, thus giving us soundness. Completeness with respect to the commutative semiring/ring theory is ensured by the validity of the normal form used (sparse Horner forms) and the equation (3). The validity of the normal forms ensure that if two polynomial expressions represent the same polynomial, then their normal form will be equal. Equation (3) ensures that the translation process from expressions to normal forms is correct. However, the equations provable by the `ring` tactic actually depend on which coefficient set is used in the types Pol and $\mathrm{PolExpr}$. By default, the `ring` tactic takes $\mathbb{Z}$ as its coefficient set, but it may not always be the best choice. For instance, for solving equations over the ring of booleans, taking the booleans themselves can let `ring` solve equations such as $\forall x : \text{Bool}, x \vee x = x$. This is because $1 + 1 = 1$ for booleans. The left side of the equality is reflected to $X + X$, whose normal form $(1 + 1) \cdot X$ is reduced to $1 \cdot X = X$. Taking the default coefficient set $\mathbb{Z}$ would not allow such equation to be provable for the booleans.

Therefore, both Frex and the `ring` tactic are sound and complete with respect to the underlying algebraic theories, but a good choice for the coefficient set of the `ring` tactic can lead to solving more equations.

5.4 Comparison

5.4.1 Supported theories

The Frex approach to algebraic simplification lets us tackle many more theories than the Rocq `ring` tactic can. In Table 1, we show examples of models for the different distributive combinations of mere/commutative mere/involutive semigroups, monoids and groups. In red are the theories that the `ring` tactic can handle. Essentially only the commutative semiring/ring corner is covered by reflection-based ring solving, whereas Frex extends to semigroup/monoid/group and involutive variants.

[2] Commutativity
[3] Involutive
[4] Commutative

Multiplicative structure			Additive structure		
Theory	**Comm.**[2]	**Inv.**[3]	**Comm. Semigroup**	**Comm. Monoid**	**Abelian Group**
Semigroup	**Mere**	**Mere**	$\mathcal{P}(\Sigma^+) \setminus \{\varnothing\}$	$\mathcal{P}(\Sigma^+)$	$\mathbb{Z}\langle\Sigma^+\rangle$
		Inv.	$\left(\mathcal{P}(\Sigma^+) \setminus \{\varnothing\}, -^R\right)$	$\left(\mathcal{P}(\Sigma^+), -^R\right)$	$(\mathbb{Z}\langle\Sigma^+\rangle, \mathrm{rev})$
	Comm.[4]	**Mere**	$2\mathbb{N}_{>0}$	$2\mathbb{N}$	$2\mathbb{Z}$
		Inv.	$\mathfrak{m}_n^\sigma \setminus \{0\}$	$\mathfrak{m}_n^\sigma$	$(2\mathbb{Z}[\boldsymbol{i}], \overline{-})$
Monoid	**Mere**	**Mere**	$\mathcal{P}(\Sigma^*) \setminus \{\varnothing\}$, $M_n(\mathbb{N}_{>0})$	$\mathcal{P}(\Sigma^*)$, $M_n(\mathbb{N})$	$M_n(\mathbb{Z})$
		Inv.	$\left(M_n(\mathbb{N}_{>0}), -^T\right)$, $\left(\mathcal{P}(\Sigma^*) \setminus \{\varnothing\}, -^R\right)$	$\left(M_n(\mathbb{N}), -^T\right)$, $\left(\mathcal{P}(\Sigma^*), -^R\right)$, $\mathrm{Rel}(X)$	$\left(M_n(\mathbb{Z}), -^T\right)$
	Comm.	**Mere**	$\mathbb{N}_{>0}$	$\mathbb{N}$	$\mathbb{Z}$, $\mathbb{R}$, $\mathbb{C}$
		Inv.	$\mathbb{N}_\sigma[\![X_1, \ldots, X_n]\!] \setminus \{0\}$	$\mathbb{N}_\sigma[\![X_1, \ldots, X_n]\!]$	$(\mathbb{C}, \overline{-})$, $\mathbb{Z}_\sigma[\![X_1, \ldots, X_n]\!]$
Group	**Mere**	**Mere**	$T_{n,\Delta_{>0}}(\mathbb{R})$	**impossible** nontrivially	**impossible** nontrivially
		Inv.	$\left(T_{n,\Delta_{>0}}(\mathbb{R}), -^\dagger\right)$	**impossible** nontrivially	**impossible** nontrivially
	Comm.	**Mere**	$\mathbb{R}_{>0}$	**impossible** nontrivially	**impossible** nontrivially
		Inv.	$(\mathbb{C}_{>0}, \overline{-})$	**impossible** nontrivially	**impossible** nontrivially

Table 1: Examples for the distributive combination of theories

The models in the table are the following:

- $\mathbb{N}, \mathbb{Z}, \mathbb{R}$: natural numbers, integers and real numbers with the usual operations.
- $(\mathbb{C}, \overline{-})$: complex numbers with conjugation.
- $\mathbb{C}_{>0} := \{a + b\boldsymbol{i} \mid a \in \mathbb{R}_{>0}, b \in \mathbb{R}\}$ are complex numbers with a strictly positive real part.
- $2\mathbb{Z}[\boldsymbol{i}] := \{2(a + b\boldsymbol{i}) \mid a, b \in \mathbb{Z}\}$ with standard addition, multiplication and conjugation as involution.
- $\mathcal{P}(\Sigma^*)$ and variants: formal languages on the alphabet Σ, with union as addition and concatenation as multiplication.
- $M_n(X)$: $n \times n$ square matrices with coefficients in X. $-^T$ denotes matrix transposition.
- $\mathbb{Z}\langle\Sigma^+\rangle$: finite integer linear combinations of nonempty words, i.e, finitely supported functions from Σ^+ to $\mathbb{Z}$. Addition is pointwise function addition and multiplication is concatenation that distributes over sums. rev is word reversal.
- $\mathrm{Rel}(X)$: the set of binary relations on X. Addition is union, and multiplication is composition. Involution is the converse $R^\dagger := \{(y, x) \mid (x, y) \in R\}$.

- $Y_\sigma[\![X_1, \ldots, X_n]\!]$: multivariate generating functions in the variables $X_1, \ldots, X_n$ with coefficients in Y. Addition and multiplication are the standard operations, the involution is defined by $f(x_1, \ldots, x_n)^{*_\sigma} := f(x_{\sigma(1)}, \ldots, x_{\sigma(n)})$ where $\sigma \in \mathfrak{S}_n$ is a permutation of order 2, i.e, $\sigma^2 = \mathrm{id}$.
- $\mathfrak{m}_n^\sigma := \{f \in \mathbb{N}_\sigma[\![X_1, \ldots, X_n]\!] \mid f(0, \ldots, 0) = 0\}$: multivariate generating functions in the variables $X_1, \ldots, X_n$ with coefficients in $\mathbb{N}$ and with no constant part.
- $\left(T_{n,\Delta_{>0}}(\mathbb{R}), -^\dagger\right)$: upper triangular matrices with coefficients > 0 on the diagonal. Addition and multiplication is standard matrix addition and multiplication. The involution $-^\dagger$ reverses the order of the coefficients on the diagonal, e.g, $\Delta(1, \ldots, n)^\dagger = \Delta(n, \ldots, 1)$.

Some combinations are impossible nontrivially. This is the case of every combination of a multiplicative group with an additive structure that has a 0 element. In such a case, every element is equal to each other. Indeed, suppose that 0 is invertible, i.e, there exists 0^{-1} such that $0 \cdot 0^{-1} = 1$. Because of the distributivity axioms, $x \vdash x \cdot 0 = 0$ holds in all of these cases. We also have the associativity of the multiplication, therefore for every element a, $a = a \cdot (0 \cdot 0^{-1}) = (a \cdot 0) \cdot 0^{-1} = 0 \cdot 0^{-1} = 1$ and every element is equal to 1. The only model validating these theories is the trivial model $\{0\}$.

5.4.2 Support for additional equalities

The Rocq `ring` tactic supports[5] taking additional equalities as arguments to solve equalities between terms. This lets one prove goals such as

```
Goal forall a b : Z, 2 * a * b = 30 -> (a + b) ^ 2 = a ^ 2 + b ^ 2 + 30.
Proof.
  intros a b H; ring [H].
Qed.
```

This is impossible with the Frex approach. The universal property of free algebras and free extensions guarantees that the only equations that are valid in the free algebra/extension are the ones that are provable with the provability relation of Figure 1. Due to the nature of the solving algorithm (§2.4), one can't provide additional facts to the solver, and therefore the goal above is not provable with Frex.

However this feature is limited. The supported additional equalities need to be of the form `m = p` where `m` is a monomial and `p` is a polynomial in the corresponding ring structure. Therefore, you can't reason about any other operators than `+` and `*`, even when giving additional axioms concerning the operator. For instance, define an abstract ring and add an involution operator `~~`. Even by giving the axioms

```
Rinv_inv : forall x, ~~ (~~ x) == x.
Rinv_antidistr : forall x y, ~~ (x * y) == ~~ y * ~~ x.
```

to the `ring` tactic, it is unable to solve the basic goal

```
Lemma invInv : forall x, ~~ (~~ x) == x.
Proof.
  intros.
  Fail ring [(Rinv_inv x) (Rinv_antidistr x)].
Abort.
```

[5] See http://rocq-prover.org/doc/V9.2.0/refman/addendum/ring.html.

because ~~ is not a part of the ring structure. This is to be expected when looking at how reflection work in §5.1. Because of this, some theories such as involutive semirings or involutive rings which are supported by Frex can't be supported by the `ring` tactic.

5.4.3 Other Frex capabilities

We have presented Frex as a generic algebraic simplification framework, but it is not limited to that role. All of the following capabilities of Frex cannot be achieved by the `ring` tactic.

By appealing to the universal property of the fral/frex, every fral/frex is isomorphic to the quotient fral/frex. This lets us extract a proof certificate, i.e, a derivation for the equation for the provability relation of Figure 1. Frex can package such a derivation as a lemma that is valid for any model of the given theory. Users can then seamlessly invoke these lemmas in any model, avoiding further Frex calls [Allais et al., 2025, Section 5.2].

The resulting derivation can also be used to pretty-print human-readable proofs, going through each step and specifying which axiom is used, and where. However, the derivation may not be optimal, leading to possibly large proofs with unnecessary steps. Figure 4 shows the first four solving steps to prove that $(3 + x) + 2 = 5 + x$, which aren't part of the optimal derivation involving only four solving steps in total. The full proof is pictured in Figure 5 in the appendix. The obtained derivation can also be used to print Idris proofs for the lemmas [Allais et al., 2025, Appendix C].

$$(3 \bullet x) \bullet [2] \underset{\uparrow}{=} (3 \bullet x) \bullet 2 \bullet [\varepsilon] \overset{\downarrow}{=} (3 \bullet x) \bullet 2 \bullet \varepsilon \bullet [\varepsilon] \underset{\uparrow}{=} (3 \bullet x) \bullet 2 \bullet \varepsilon \bullet \varepsilon \bullet [\varepsilon] \overset{\downarrow}{=} \cdots = 5 \bullet x$$

⟨ Right neutrality] ⟨ Left neutrality] ⟨ Left neutrality] ⟨ Left neutrality]

Figure 4: Extract of a Frex-extracted proof of $(3 \bullet x) \bullet 2 = 5 \bullet x$ in the additive monoid over natural numbers.

Frex can also be used to normalise code. Corbyn et al. [2022] present a normalisation by evaluation framework using Frex that can simplify higher-order functional programs with algebraic structure. For instance, the framework can simplify simply typed lambda calculus terms extended with an arbitrary commutative monoid M. Take M to be the commutative monoid of integers with multiplication and Frex can normalise terms such as $\lambda x.(2 * 3) * x$ or $\lambda x.(\lambda y.2 * y)(x * 3)$ to $\lambda x.6 * x$, combining beta-reduction and algebraic simplification. While a naive normaliser suffices to reduce the first term to its normal, reducing the second term requires reordering the terms and invoking the axioms of the commutative monoid, which requires Frex.

The Frex simplification can also be applied to multi-stage programming. In Yallop et al. [2018, Section 4.3], Frex is used to simplify the code generated by a staged version of `sprintf` to produce more efficient code requiring less string concatenations, reducing expressions such as `((("" ++ show x) ++ "a") ++ "b") ++ show y` to `show x ++ ("ab" ++ show y)`.

6 Conclusion and prospects

We have found a way to modularly combine free algebras for component theories to get a free algebra for distributive combinations of theories, letting us use the work of Allais et al. [2025] to modularly obtain solvers for varieties of semirings. From monoids and commutative monoids we recover semiring solvers, and from existing involutive constructions we additionally obtain

involutive semiring-like solvers. Already 28 non-trivial solvers can be implemented, and performance evaluation indicates that such solvers are suited for use in dependently-typed programming languages.

The use of category theory here is semi-implicit, a slightly different result is stated here to keep this report self-contained. The actual construction uses 2-category and especially 2-functors to transport adjunctions, as well as operads and multicategories to capture the distributivity of the multiplicative operations over the additive operations.

It remains to find a construction that works for free extensions. We proved that a modular construction reusing free extensions for the component theories exist. We also managed to narrow down our scope of research to searching for a multicategory suited for free extensions and an adjunction.

Other than giving a construction for involutive free extensions, Allais et al. [2023] also presents the theory of multi-frex: free extensions of multi-sorted theories. It allows us to deal with a lot of other theories, such as non-square matrices or linear transformations. However, it has yet to be formalised in Idris. Doing so would let us explore other distributive theories, such as modules, a generalisation of vector spaces in which the field of scalars is replaced by a ring. Modules can be modeled as the distributive combination of said ring over an abelian group.

Another axis of development for the Frex project is the support for second-order/parametrised and essentially algebraic theories. No progress has been made in that direction yet.

Recently, Fujii et al. [2026] gave a canonical construction of a distributive law under some hypothesis in substructural contexts. A future direction is to systematically compare with their work.

References

Allais, G., Brady, E., Corbyn, N., Kammar, O., & Yallop, J. [2025]. Frex: Dependently typed algebraic simplification. *Proceedings of the ACM on Programming Languages*, *9*[ICFP], 30–65. https://doi.org/10.1145/3747506

Allais, G., Corbyn, N., Lindley, S., & Yallop, J. [2023]. Modular construction of multi-sorted free extensions (short paper). https://api.semanticscholar.org/CorpusID:260005086

Beck, J. [1969]. Distributive laws. In B. Eckmann [Ed.], *Seminar on triples and categorical homology theory* [pp. 119–140]. Springer Berlin Heidelberg.

Bishop, E. [1967]. *Foundations of constructive analysis*. Mcgraw-Hill.

Burris, S., & Sankappanavar, H. [1981, January]. *A course in universal algebra* [Vol. 91]. https://doi.org/10.2307/2322184

Corbyn, N. [2021]. *Proof synthesis with free extensions in intensional type theory* [tech. rep.] [MEng Dissertation]. University of Cambridge.

Corbyn, N., Kammar, O., Lindley, S., Valliappan, N., & Yallop, J. [2022]. Normalization by evaluation with free extensions.

Fujii, S., Tsai, Y. C., Montacute, Y., & Hasuo, I. [2026]. Monads and Distributive Laws in Substructural Contexts. In C. Faggian & J.-P. Katoen [Eds.], *41st annual symposium on logic in computer science (lics 2026)* [45:1–45:28, Vol. 380]. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. https://doi.org/10.4230/LIPIcs.LICS.2026.45
Keywords: Monad, distributive law, operad, category theory, effect.

Grégoire, B., & Mahboubi, A. [2005]. Proving equalities in a commutative ring done right in coq. In J. Hurd & T. Melham [Eds.], *Theorem proving in higher order logics* [pp. 98–113]. Springer Berlin Heidelberg.

Hofmann, M. [1997]. *Extensional constructs in intensional type theory* [C. J. Rijsbergen, Ed.]. Springer-Verlag.

Hyland, M., & Power, J. [2006]. Discrete lawvere theories and computational effects [Algebra and Coalgebra in Computer Science]. *Theoretical Computer Science*, *366*[1], 144–162. https://doi.org/https://doi.org/10.1016/j.tcs.2006.07.007

Kidney, D. O. [2019]. Automatically and efficiently illustrating polynomial equalities in agda. *URL: https://doisinkidney. com/pdfs/bsc-thesis. pdf*.

MacLane, S. [1971]. *Categories for the working mathematician* [Graduate Texts in Mathematics, Vol. 5]. Springer-Verlag.

Nielsen, J. [1994]. *Usability engineering*. Morgan Kaufmann Publishers Inc.

Varacca, D., & Winskel, G. [2006]. Distributing probability over non-determinism. *Mathematical Structures in Computer Science*, *16*[1], 87–113. https://doi.org/10.1017/S0960129505005074

Yallop, J., von Glehn, T., & Kammar, O. [2018]. Partially-static data as free extension of algebras. *Proc. ACM Program. Lang.*, *2*[ICFP]. https://doi.org/10.1145/3236795

Zwart, M., & Marsden, D. [2018]. Don't try this at home: No-go theorems for distributive laws. https://arxiv.org/abs/1811.06460

Appendices

A Additional material

probably find a better name

Proof (Proposition 1)

We want to prove that $A[\![l]\!]\,\mathrm{env}_A = A[\![r]\!]\,\mathrm{env}_A$. The universal property of FX gives a homomorphism $h : \underline{FX} \to \underline{A}$ such that $h \circ \mathrm{env}_{FX} = \mathrm{env}_A$.

Lemma 5 (Homomorphisms preserve term semantics)

Let A, B be two $\mathcal{T}$-algebras, $X \vdash t$ be a term and $h : A \to B$ be a $\mathcal{T}$-homomorphism. Then for every function $\mathrm{env} : X \to \underline{A}$, $h\big(A[\![l]\!]\,\mathrm{env}\big) = B[\![r]\!]\,(h \circ \mathrm{env})$.

Proof

This is a simple induction, since h preserves the semantics of the operators in $\mathcal{T}$. □

Lemma 6 (Homomorphisms preserve equations)

Let A, B be two $\mathcal{T}$-algebras, $\mathrm{env} : X \to \underline{A}$ be a function, $X \vdash l = r$ be an equation and $h : A \to B$ be a $\mathcal{T}$-homomorphism. Suppose that $A[\![l]\!]\,\mathrm{env} = A[\![r]\!]\,\mathrm{env}$.

Then $B[\![l]\!]\,(h \circ \mathrm{env}) = B[\![r]\!]\,(h \circ \mathrm{env})$.

Proof

$$\begin{aligned} B[\![l]\!]\,(h \circ \mathrm{env}) &= h\big(A[\![l]\!]\,\mathrm{env}\big) && \text{since } h \text{ preserves term semantics} \\ &= h\big(A[\![r]\!]\,\mathrm{env}\big) && \text{since } A[\![l]\!]\,\mathrm{env} = A[\![r]\!]\,\mathrm{env} \text{ by hypothesis} \\ &= B[\![r]\!]\,(h \circ \mathrm{env}) && \text{since } h \text{ preserves term semantics} \end{aligned}$$

□

Now we can finally prove our equality in A. FX validates eq, so $FX[\![l]\!]\,\mathrm{env}_{FX} = FX[\![r]\!]\,\mathrm{env}_{FX}$. Since h is a homomorphism and homomorphisms preserve equations, $A[\![l]\!]\,(h \circ \mathrm{env}_{FX}) = A[\![r]\!]\,(h \circ \mathrm{env}_{FX})$. But $h \circ \mathrm{env}_{FX} = \mathrm{env}_A$, therefore $A[\![l]\!]\,\mathrm{env}_A = A[\![r]\!]\,\mathrm{env}_A$. □

Definition 7 (Abelian group monad)

The Abelian group monad $\langle A, \eta^A, \mu^A \rangle$ is given by:

- $A : \mathbf{Set} \to \mathbf{Set}$ maps a set X to the set containing all finite multisets with elements taken from X and with multiplicities in the integers (i.e, elements of the multiset are allowed to have a negative multiplicity).
- $\eta^A_X : X \to AX$ maps an element $x \in X$ to the singleton $\{x : 1\}$.
- $\mu^A_X : AAX \to AX$ takes a union of multisets, accounting for all multiplicities.

$$\mu^A_X \{\{a : 1, b : -2\} : 1, \{b : 1\} : 3\} = \{a : (1 \times 1), b : (-2 \times 1 + 1 \times 3)\} = \{a : 1, b : 1\}$$

Definition 8 (Barycentric algebras)

The theory of barycentric algebras $\mathbf{BA}$ consists of a binary operator $\oplus_p$ for every $p \in [0, 1]$ and the axioms

$$x \vdash x \oplus_p x = x \qquad \text{idempotence}$$

$$x, y \vdash x \oplus_p y = y \oplus_{1-p} x \qquad \text{skew commutativity}$$

$$x, y, z \vdash x \oplus_p (y \oplus_r z) = \left(x \oplus_{\frac{p}{p+(1-p)r}} y\right) \oplus_{p+(1-p)r} z \qquad \text{skew associativity}$$

Definition 9 (Join semi-lattices)

The theory of join semi-lattices $\mathbf{SL}$ consists of a binary operator $\vee$ and a constant $\bot$, along with the axioms

$$x \vdash x \vee \bot = x \qquad \text{unit}$$

$$x, y, z \vdash (x \vee y) \vee z = x \vee (y \vee z) \qquad \text{associativity}$$

$$x, y \vdash x \vee y = y \vee x \qquad \text{commutativity}$$

$$x \vdash x \vee x = x \qquad \text{idempotence}$$

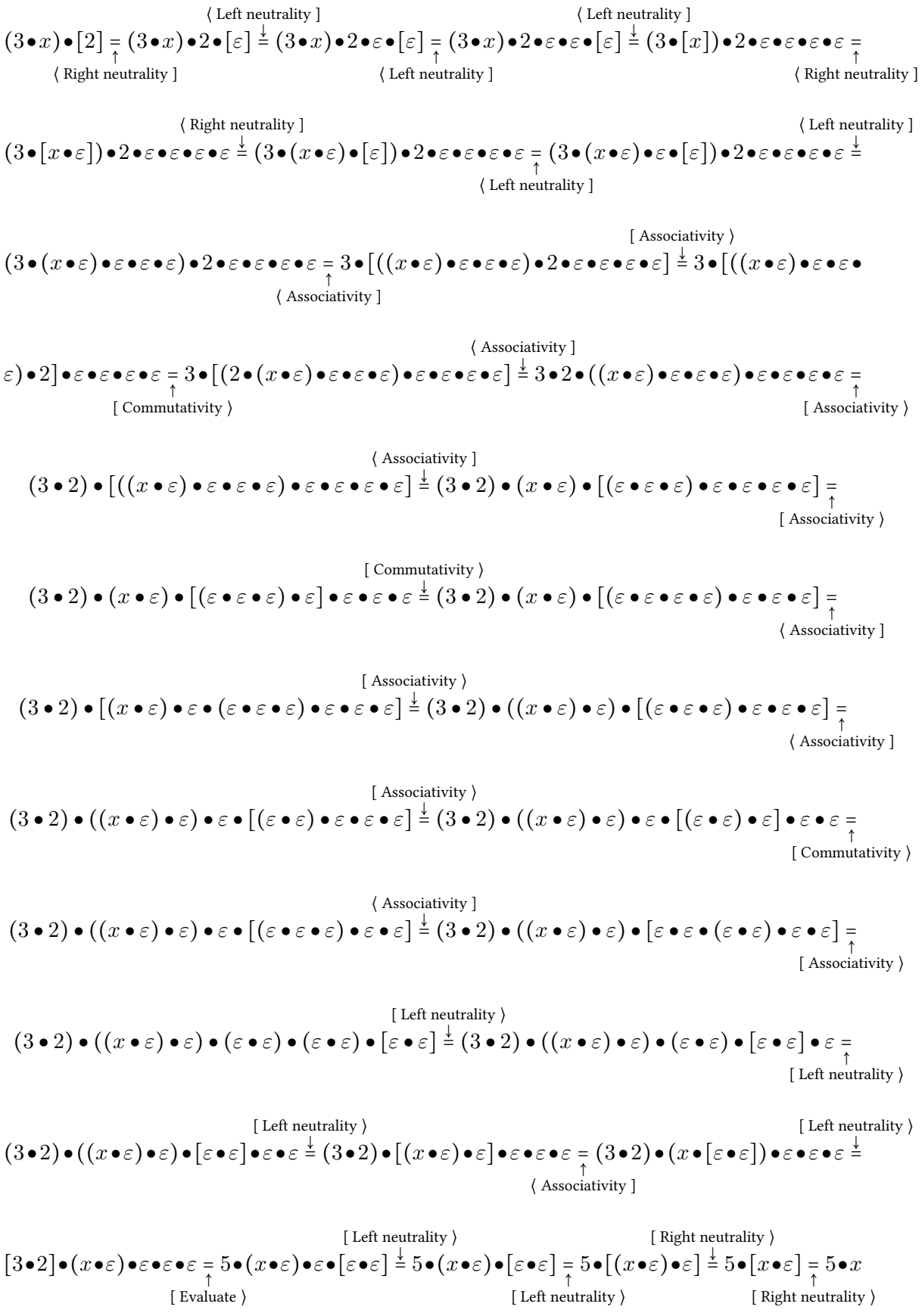


Figure 5: Frex-extracted proof of $(3 \bullet x) \bullet 2 = 5 \bullet x$ in the additive monoid over natural numbers.

B Internship context

TODO

C Source code

The source code can be found on a fork of the Idris Frex project on GitHub at https://github.com/S-c-r-a-t-c-h-y/idris-frex/tree/amaury.

D Proof of Theorem 4

The full proof of Theorem 4 exceeds 50 pages and is compiled in a single file on GitHub at https://github.com/S-c-r-a-t-c-h-y/semiring-solvers/blob/main/proof/semirings.pdf.

E Research notes

Research notes are available online for completeness on GitHub at https://github.com/S-c-r-a-t-c-h-y/semiring-solvers/blob/main/notes/notes.pdf.

Acknowledgments

TODO